

TRABAJO PRACTICO N°2- ESTRUCTURA DE DATOS

Alumno : Lucas Gaiton

Profesor : Reinaga Hector

Indice

1. Dado un arreglo con los valores: (2, 124, 23, 5, 89, -1, 44, 643, 34, 76, 43, 0) ordenarlo “a mano” utilizando los siguientes métodos:
2 b) Heapsort.
..... 2
6. Dado un vector cualquiera, que contiene objetos con distintos atributos, defina un algoritmo que implemente el método Mergesort sobre el vector, para cualquiera de los atributos del mismo. El método debe aceptar como parámetros el vector, el número del atributo por el cual se quiere ordenar (clave) y el orden que se desea (ascendente o descendente)
15

1. Dado un arreglo con los valores: (2, 124, 23, 5, 89, -1, 44, 643, 34, 76, 43, 0) ordenarlo “a mano” utilizando los siguientes métodos:

Trabajo practico N°2 – Estructura de datos

b) Heapsort.

Paso 1 : convertir el array en montículo

Array original

(2, 124, 23, 5, 89, -1, 44, 643, 34, 76, 43, 0)

2	124	23	5	89	-1	44	643	34	76	43	0
---	-----	----	---	----	----	----	-----	----	----	----	---

Comienza la creación del montículo:

Ingreso el primer elemento:

2

Ingreso el 124:

2	124
---	-----

Se restaura el montículo hacia arriba

2	124
---	-----



124	2
-----	---

Ingreso el 23:

124	2	23
-----	---	----

Ingresar el 5:

124	2	23	5
-----	---	----	---

Se restaura el montículo hacia arriba:

124	2	23	5
-----	---	----	---



124	5	23	2
-----	---	----	---

Trabajo practico N°2 – Estructura de datos

Ingresa el 89:

124	5	23	2	89
-----	---	----	---	----

Se restaura el montículo hacia arriba:

124	5	23	2	89
-----	---	----	---	----



124	89	23	2	5
-----	----	----	---	---

Ingresa el -1

124	89	23	2	5	-1
-----	----	----	---	---	----

Ingreso de 44

124	89	23	2	5	-1	44
-----	----	----	---	---	----	----

Se restaura el montículo hacia arriba

124	89	23	2	5	-1	44
-----	----	----	---	---	----	----



124	89	44	2	5	-1	23
-----	----	----	---	---	----	----

Ingresa el 643:

124	89	44	2	5	-1	23	643
-----	----	----	---	---	----	----	-----

Se restaura el montículo hacia arriba

124	89	44	2	5	-1	23	643
-----	----	----	---	---	----	----	-----



124	89	44	643	5	-1	23	2
-----	----	----	-----	---	----	----	---



124	643	44	89	5	-1	23	2
-----	-----	----	----	---	----	----	---



Trabajo practico N°2 – Estructura de datos

643	124	44	89	5	-1	23	2
-----	-----	----	----	---	----	----	---

Ingresa el 34:

643	124	44	89	5	-1	23	2	34
-----	-----	----	----	---	----	----	---	----

Ingresa el 76:

643	124	44	89	5	-1	23	2	34	76
-----	-----	----	----	---	----	----	---	----	----

Se restaura el montículo hacia arriba

643	124	44	89	5	-1	23	2	34	76
-----	-----	----	----	---	----	----	---	----	----



643	124	44	89	76	-1	23	2	34	5
-----	-----	----	----	----	----	----	---	----	---

Ingresa el 43:

643	124	44	89	76	-1	23	2	34	5	43
-----	-----	----	----	----	----	----	---	----	---	----

Ingresa el 0:

643	124	44	89	76	-1	23	2	34	5	43	0
-----	-----	----	----	----	----	----	---	----	---	----	---

Paso 2

Se construye un array con los elementos del montículo de atrás hacia delante:

Montículo:

Se saca el 643:

643	124	44	89	76	-1	23	2	34	5	43	0
-----	-----	----	----	----	----	----	---	----	---	----	---

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

0	124	44	89	76	-1	23	2	34	5	43	643
---	-----	----	----	----	----	----	---	----	---	----	-----

Se elimina el último elemento del array

0	124	44	89	76	-1	23	2	34	5	43
---	-----	----	----	----	----	----	---	----	---	----

Se restaura el montículo hacia abajo:

0	124	44	89	76	-1	23	2	34	5	43
---	-----	----	----	----	----	----	---	----	---	----

Trabajo practico N°2 – Estructura de datos



124	0	44	89	76	-1	23	2	34	5	43
-----	---	----	----	----	----	----	---	----	---	----



124	89	44	0	76	-1	23	2	34	5	43
-----	----	----	---	----	----	----	---	----	---	----



124	89	44	34	76	-1	23	2	0	5	43
-----	----	----	----	----	----	----	---	---	---	----

Nuevo array:

643

Se saca el 124:

124	89	44	34	76	-1	23	2	0	5	43
-----	----	----	----	----	----	----	---	---	---	----

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

43	89	44	34	76	-1	23	2	0	5	124
----	----	----	----	----	----	----	---	---	---	-----

Se elimina el último elemento del array

43	89	44	34	76	-1	23	2	0	5
----	----	----	----	----	----	----	---	---	---

Se restaura el montículo hacia abajo:

43	89	44	34	76	-1	23	2	0	5
----	----	----	----	----	----	----	---	---	---



89	76	44	34	43	-1	23	2	0	5
----	----	----	----	----	----	----	---	---	---

Nuevo array:

124	643
-----	-----

Se saca el 89:

89	76	44	34	43	-1	23	2	0	5
----	----	----	----	----	----	----	---	---	---

Trabajo practico N°2 – Estructura de datos

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

5	76	44	34	43	-1	23	2	0	89
---	----	----	----	----	----	----	---	---	----

Se elimina el último elemento del array

5	76	44	34	43	-1	23	2	0
---	----	----	----	----	----	----	---	---

Se restaura el montículo hacia abajo:

5	76	44	34	43	-1	23	2	0
---	----	----	----	----	----	----	---	---



76	43	44	34	5	-1	23	2	0
----	----	----	----	---	----	----	---	---

Nuevo array:

89	124	643
----	-----	-----

Se saca el 76:

76	43	44	34	5	-1	23	2	0
----	----	----	----	---	----	----	---	---

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

0	43	44	34	5	-1	23	2	76
---	----	----	----	---	----	----	---	----

Se elimina el último elemento del array

0	43	44	34	5	-1	23	2
---	----	----	----	---	----	----	---

Se restaura el montículo hacia abajo:

0	43	44	34	5	-1	23	2
---	----	----	----	---	----	----	---



44	43	0	34	5	-1	23	2
----	----	---	----	---	----	----	---



44	43	23	34	5	-1	0	2
----	----	----	----	---	----	---	---

Trabajo practico N°2 – Estructura de datos

Nuevo array:

76	89	124	643
----	----	-----	-----

Se saca el 44:

44	43	23	34	5	-1	0	2
----	----	----	----	---	----	---	---

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

2	43	23	34	5	-1	0	44
---	----	----	----	---	----	---	----

Se elimina el último elemento del array

2	43	23	34	5	-1	0
---	----	----	----	---	----	---

Se restaura el montículo hacia abajo:

43	2	23	34	5	-1	0
----	---	----	----	---	----	---



43	34	23	2	5	-1	0
----	----	----	---	---	----	---

Nuevo array:

44	76	89	124	643
----	----	----	-----	-----

Se saca el 43:

43	34	23	2	5	-1	0
----	----	----	---	---	----	---

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

0	34	23	2	5	-1	43
---	----	----	---	---	----	----

Se elimina el último elemento del array

0	34	23	2	5	-1
---	----	----	---	---	----

Se restaura el montículo hacia abajo:

0	34	23	2	5	-1
---	----	----	---	---	----

Trabajo practico N°2 – Estructura de datos



34	0	23	2	5	-1
----	---	----	---	---	----



34	5	23	2	0	-1
----	---	----	---	---	----

Nuevo array:

43	44	76	89	124	643
----	----	----	----	-----	-----

Se saca el 34:

34	5	23	2	0	-1
----	---	----	---	---	----

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

-1	5	23	2	0	34
----	---	----	---	---	----

Se elimina el último elemento del array

-1	5	23	2	0
----	---	----	---	---

Se restaura el montículo hacia abajo:

-1	5	23	2	0
----	---	----	---	---



23	5	-1	2	0
----	---	----	---	---

Nuevo array:

34	43	44	76	89	124	643
----	----	----	----	----	-----	-----

Se saca el 23:

23	5	-1	2	0
----	---	----	---	---

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

Trabajo practico N°2 – Estructura de datos

0	5	-1	2	23
---	---	----	---	----

Se elimina el último elemento del montículo:

0	5	-1	2
---	---	----	---

Se restaura el montículo hacia abajo:

0	5	-1	2
---	---	----	---



5	0	-1	2
---	---	----	---



5	2	-1	0
---	---	----	---

Nuevo array:

23	34	43	44	76	89	124	643
----	----	----	----	----	----	-----	-----

Se saca el 23:

5	2	-1	0
---	---	----	---

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

0	2	-1	5
---	---	----	---

Se elimina el último elemento del montículo:

0	2	-1
---	---	----

Se restaura el montículo hacia abajo:

0	2	-1
---	---	----



Trabajo practico N°2 – Estructura de datos

2	0	-1
---	---	----

Nuevo array:

5	23	34	43	44	76	89	124	643
---	----	----	----	----	----	----	-----	-----

Se saca el 2:

2	0	-1
---	---	----

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

-1	0	2
----	---	---

Se elimina el último elemento del montículo:

-1	0
----	---

Se restaura el montículo hacia abajo:

-1	0
----	---



0	-1
---	----

Nuevo array:

2	5	23	34	43	44	76	89	124	643
---	---	----	----	----	----	----	----	-----	-----

Se saca el 0:

0	-1
---	----

Se intercambia de lugar el último elemento del montículo con el primero (índice 0) (Raíz del árbol)

-1	0
----	---

Se elimina el último elemento del montículo:

Trabajo practico N°2 – Estructura de datos

-1

Nuevo array:

0	2	5	23	34	43	44	76	89	124	643
---	---	---	----	----	----	----	----	----	-----	-----

Se saca el -1:

-1

Nuevo array:

-1	0	2	5	23	34	43	44	76	89	124	643
----	---	---	---	----	----	----	----	----	----	-----	-----

Resultado:

-1	0	2	5	23	34	43	44	76	89	124	643
----	---	---	---	----	----	----	----	----	----	-----	-----

e) Shakersort.:

Array original:

2	124	23	5	89	-1	643	34	76	0
---	-----	----	---	----	----	-----	----	----	---

Se realiza la primera iteración de derecha a izquierda.

2	124	23	5	89	-1	643	34	76	0
---	-----	----	---	----	----	-----	----	----	---

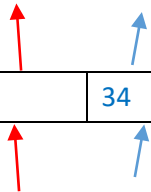
2	124	23	5	89	-1	643	34	0	76
---	-----	----	---	----	----	-----	----	---	----



2	124	23	5	89	-1	643	34	0	76
---	-----	----	---	----	----	-----	----	---	----



2	124	23	5	89	-1	643	0	34	76
---	-----	----	---	----	----	-----	---	----	----



Trabajo practico N°2 – Estructura de datos

2	124	23	5	89	-1	643	0	34	76
---	-----	----	---	----	----	-----	---	----	----

2	124	23	5	89	-1	0	643	34	76
---	-----	----	---	----	----	---	-----	----	----

2	124	23	5	89	-1	0	643	34	76
---	-----	----	---	----	----	---	-----	----	----

2	124	23	5	89	-1	0	643	34	76
---	-----	----	---	----	----	---	-----	----	----

2	124	23	5	-1	89	0	643	34	76
---	-----	----	---	----	----	---	-----	----	----

2	124	23	5	-1	89	0	643	34	76
---	-----	----	---	----	----	---	-----	----	----

2	124	23	-1	5	89	0	643	34	76
---	-----	----	----	---	----	---	-----	----	----

2	124	23	-1	5	89	0	643	34	76
---	-----	----	----	---	----	---	-----	----	----

2	124	-1	23	5	89	0	643	34	76
---	-----	----	----	---	----	---	-----	----	----

2	-1	124	23	5	89	0	643	34	76
---	----	-----	----	---	----	---	-----	----	----

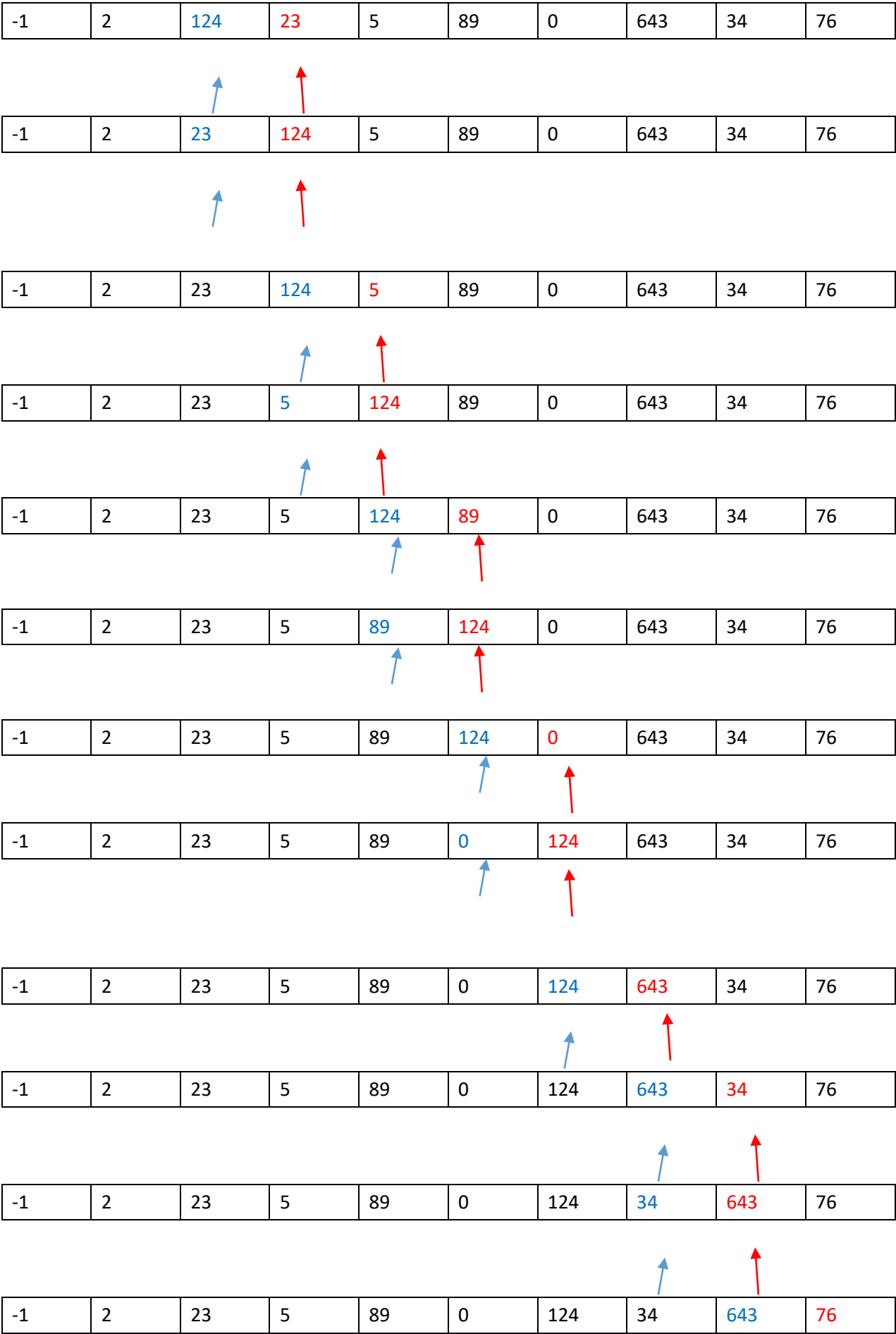
-1	2	124	23	5	89	0	643	34	76
----	---	-----	----	---	----	---	-----	----	----

Se realiza la primera iteración de izquierda a la derecha:

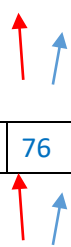
-1	2	124	23	5	89	0	643	34	76
----	---	-----	----	---	----	---	-----	----	----

-1	2	124	23	5	89	0	643	34	76
----	---	-----	----	---	----	---	-----	----	----

Trabajo practico N°2 – Estructura de datos



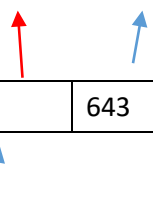
Trabajo practico N°2 – Estructura de datos



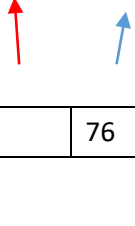
-1	2	23	5	89	0	124	34	76	643
----	---	----	---	----	---	-----	----	----	-----

Se realiza la segunda iteración de derecha a izquierda.

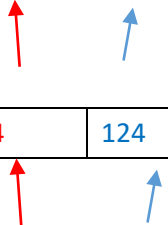
-1	2	23	5	89	0	124	34	76	643
----	---	----	---	----	---	-----	----	----	-----



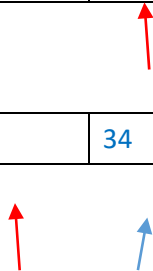
-1	2	23	5	89	0	124	34	76	643
----	---	----	---	----	---	-----	----	----	-----



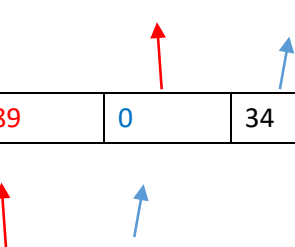
-1	2	23	5	89	0	124	34	76	643
----	---	----	---	----	---	-----	----	----	-----



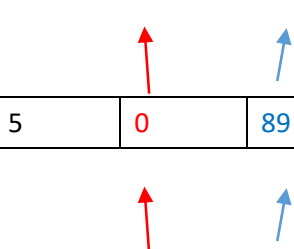
-1	2	23	5	89	0	34	124	76	643
----	---	----	---	----	---	----	-----	----	-----



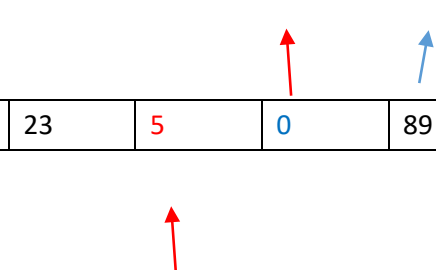
-1	2	23	5	89	0	34	124	76	643
----	---	----	---	----	---	----	-----	----	-----



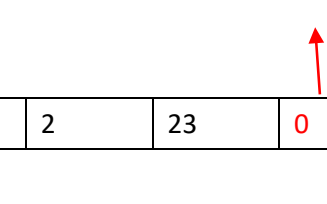
-1	2	23	5	89	0	34	124	76	643
----	---	----	---	----	---	----	-----	----	-----



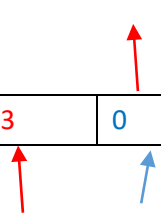
-1	2	23	5	0	89	34	124	76	643
----	---	----	---	---	----	----	-----	----	-----



-1	2	23	5	0	89	34	124	76	643
----	---	----	---	---	----	----	-----	----	-----



-1	2	23	0	5	89	34	124	76	643
----	---	----	---	---	----	----	-----	----	-----



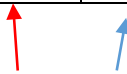
-1	2	23	0	5	89	34	124	76	643
----	---	----	---	---	----	----	-----	----	-----

Trabajo practico N°2 – Estructura de datos

-1	2	0	23	5	89	34	124	76	643
----	---	---	----	---	----	----	-----	----	-----



-1	2	0	23	5	89	34	124	76	643
----	---	---	----	---	----	----	-----	----	-----



-1	0	2	23	5	89	34	124	76	643
----	---	---	----	---	----	----	-----	----	-----

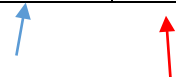


Se realiza la segunda iteración de izquierda a derecha:

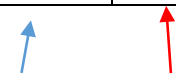
-1	0	2	23	5	89	34	124	76	643
----	---	---	----	---	----	----	-----	----	-----



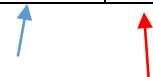
-1	0	2	23	5	89	34	124	76	643
----	---	---	----	---	----	----	-----	----	-----



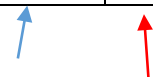
-1	0	2	5	23	89	34	124	76	643
----	---	---	---	----	----	----	-----	----	-----



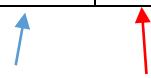
-1	0	2	5	23	89	34	124	76	643
----	---	---	---	----	----	----	-----	----	-----



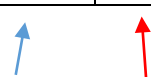
-1	0	2	5	23	34	89	124	76	643
----	---	---	---	----	----	----	-----	----	-----



-1	0	2	5	23	34	89	124	76	643
----	---	---	---	----	----	----	-----	----	-----

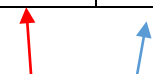


-1	0	2	5	23	34	89	76	124	643
----	---	---	---	----	----	----	----	-----	-----



Se realiza la segunda iteración de derecha a izquierda:

-1	0	2	5	23	34	89	76	124	643
----	---	---	---	----	----	----	----	-----	-----



-1	0	2	5	23	34	89	76	124	643
----	---	---	---	----	----	----	----	-----	-----



-1	0	2	5	23	34	76	89	124	643
----	---	---	---	----	----	----	----	-----	-----

Trabajo practico N°2 – Estructura de datos



Se realiza la segunda iteración de izquierda a derecha:

-1	0	2	5	23	34	76	89	124	643
----	---	---	---	----	----	----	----	-----	-----



Se realiza la segunda iteración de derecha a izquierda:

-1	0	2	5	23	34	76	89	124	643
----	---	---	---	----	----	----	----	-----	-----



Finaliza el ordenamiento.

6. Dado un vector cualquiera, que contiene objetos con distintos atributos, defina un algoritmo que implemente el método Mergesort sobre el vector, para cualquiera de los atributos del mismo. El método debe aceptar como parámetros el vector, el número del atributo por el cual se quiere ordenar (clave) y el orden que se desea (ascendente o descendente).

Desarrollo:

```
public class TpN2eje6 {
    public static void mergeSort(Object[] vec, String atributo,
boolean ascendente) {
        Object[] tmp = new Object[vec.length];
        mergeSort(vec, tmp, 0, vec.length - 1, atributo,
ascendente);
    }
    private static void mergeSort(Object[] vec, Object[] tmp, int
left, int right, String atributo,
boolean ascendente) {
        if (left < right) {
            int center = (left + right) / 2;
            mergeSort(vec, tmp, left, center, atributo,
ascendente);
            mergeSort(vec, tmp, center + 1, right, atributo,
ascendente);
            merge(vec, tmp, left, center, right, atributo,
ascendente);
        }
    }
}
```

```
private static void merge(Object[] vec, Object[] tmp, int
left, int center, int right, String atributo,
    boolean ascendente) {
    int aptr = left;
    int bptr = center + 1;
    int cptr = left;

    while (aptr <= center && bptr <= right) {
        int comparison = compareByAttribute(vec[aptr],
vec[bptr], atributo);
        if (ascendente) {
            if (comparison <= 0) {
                tmp[cptr++] = vec[aptr++];
            } else {
                tmp[cptr++] = vec[bptr++];
            }
        } else {
            if (comparison >= 0) {
                tmp[cptr++] = vec[aptr++];
            } else {
                tmp[cptr++] = vec[bptr++];
            }
        }
    }
    while (aptr <= center) {
        tmp[cptr++] = vec[aptr++];
    }
    while (bptr <= right) {
        tmp[cptr++] = vec[bptr++];
    }
    for (int i = left; i <= right; i++) {
        vec[i] = tmp[i];
    }
}

// Método que compara dos objetos según el atributo dado
usando reflexión
private static int compareByAttribute(Object o1, Object o2,
String atributo) {
    try {
        // Obtener el campo (atributo) por su nombre
```

Trabajo practico N°2 – Estructura de datos

```
        Field field =  
o1.getClass().getDeclaredField(atributo);  
        field.setAccessible(true); // Permite acceso a campos  
privados  
  
        // Obtener los valores del atributo  
        Comparable value1 = (Comparable) field.get(o1);  
        Comparable value2 = (Comparable) field.get(o2);  
  
        // Comparar los valores usando el método compareTo  
        return value1.compareTo(value2);  
    } catch (Exception e) {  
        e.printStackTrace();  
        return 0;  
    }  
}
```